

Ingegneria del Software

Mattia Masina

CdL in Informatica – Università di Bologna

Anno Accademico 2026/2027

Indice

1	Introduzione e Contesto: La Produzione del Software	3
1.1	La Trasformazione Digitale e il Contesto Industriale	3
1.2	Definizioni, Classificazioni e Requisiti	3
1.3	Dipendenze ed Ecosistemi Software	5
1.4	Aspetti Legali, Economici e Garanzie	5
1.5	Rischi e Qualità del Software	5
1.6	Evoluzione dell'AI e Visione della Disciplina	6
1.7	Risorse Utili	7
2	Essence	9
2.1	Origini della Disciplina e Iniziativa SEMAT	9
2.2	Cos'è Essence e i 7 Alpha del Kernel	10
2.3	Strumenti Pratici, Metodologie e Formazione	12
2.4	Metodi, Pratiche e Ciclo di Vita degli Alpha	14
2.5	Elementi Complementari e Studio di Caso (UniMagic)	17

1 Introduzione e Contesto: La Produzione del Software

1.1 La Trasformazione Digitale e il Contesto Industriale

La trasformazione digitale consiste nel **far vivere meglio le persone grazie al software**. Oggi ci sono **oltre 180 milioni di sviluppatori** su GitHub (dato 2025), con una crescita di 36 milioni nell'ultimo anno; l'80% dei nuovi iscritti adotta GitHub Copilot entro la prima settimana.

Leggi Fondamentali

- **Legge di Wirth:** “Il software diventa lento più rapidamente di quanto l'hardware diventi più veloce”. Oggi: il codice si moltiplica più rapidamente della nostra capacità di comprenderlo.
- **Legge di Eagleson:** “Qualsiasi codice che non hai rivisto negli ultimi sei mesi ti sembrerà estraneo”. Oggi: il codice generato da LLM non rivisto riga per riga ti sembrerà tuo finché non si rompe.

Il Software come Prodotto Industriale. Il software è **sempre il prodotto di un processo di sviluppo** che inizia con un'idea e termina con il ritiro. L'industria mondiale cresce al 5–10% annuo. Il costo di sviluppo tende a crescere con il **quadrato delle dimensioni** (legge classica); oggi gli assistenti AI abbassano il costo per riga ma non quello di capire, integrare e mantenere sistemi complessi.

Cifre Chiave (SLOC – Source Lines of Code)

Prodotto	SLOC
NASA Space Shuttle Flight Control	1,8M
Linux Kernel 4.2 (2016)	20,2M
Windows NT 5.0 (1998)	20M
macOS 10.4 (2005)	86M
Debian 7.0 (2012)	419M
Google (monorepo, 2025)	2 miliardi

Red Hat Linux 7.1 (2000): 30M SLOC, 8.000 persone-anno (modello COCOMO), costo stimato >1 miliardo \$.

1.2 Definizioni, Classificazioni e Requisiti

Cosa è il Software: Quattro Prospettive

1. **Prodotto invisibile, intangibile, duplicabile**, costoso: opera dell'ingegno protetta da copyright.
2. **Servizio digitale:** ha un'interfaccia (API), basa su infrastruttura hw/sw/virtuale,

garantisce QoS via contratto.

3. **Componente di sistema:** può essere *off-the-shelf* (larga diffusione) o *custom* (commissionato).
4. **Macchina astratta:** trasforma il computer realizzando funzioni utili; ha un'architettura (struttura + comportamento).

Categorie di Software (2025):

- **Apps ed Ecosistemi Software** (App Store, Play Store, AWS Marketplace)
- **Software as a Service (SaaS)** – es. Google, Chrome
- **Embedded Software / IoT** – “nascosto” negli oggetti
- **Social Software (Web 2.0)** – Facebook, X, LinkedIn, wiki, forum
- **Software Libero (Open Source)** – 4 libertà FSF (uso, studio, distribuzione, miglioramento)
- **API Economy** – Amazon, Philips Hue, Google Maps, LLM APIs (OpenAI, Anthropic)
- **Agenti e Orchestratori AI** – entità autonome + coordinatori di flussi multi-agente

Classificazione per Tipo di Prodotto: Tre Categorie

- **Prodotti Generici (OTS – Off The Shelf):** creati da un produttore, venduti a molti clienti (es. videogiochi, Office).
- **Sistemi Commissionati (Custom):** sviluppati per un cliente specifico (es. portale universitario).
- **Servizi in Perpetuo Sviluppo:** sistemi 24/7 in continuo cambiamento (es. Facebook, Amazon, chatbot).

Distinzione Fondamentale: Requisiti vs Feature

- **Requisito:** funzione o qualità *testabile* che l'implementazione deve possedere. *Importante per il cliente.*
- **Feature:** insieme di funzioni che permettono l'uso del prodotto in un servizio. *Importante per il fornitore.*

Esempio e-commerce: Requisiti = registrazione, carrello, indirizzi, pagamento. Feature = “Carrello per negozio elettronico” (es. OpenCart).

1.3 Dipendenze ed Ecosistemi Software

Grafo delle Dipendenze. Ogni prodotto software dipende da altri prodotti, che a loro volta dipendono da altri. Si associa a ciascun sistema un **grafo di dipendenze** i cui nodi sono **pacchetti software (librerie) in versioni specifiche**. La gestione delle dipendenze è centrale per sicurezza, riproducibilità e manutenibilità.

Ecosistemi Software. Mercati dove si vendono prodotti (App Store) o componenti/servizi (AWS). Caratteristica: collezione di prodotti su piattaforma definita da un'azienda, che si sviluppano ed evolvono nello stesso ambiente.

- App Store 2025: 117 miliardi \$ ricavi vs 49 miliardi \$ Google Play; 142 miliardi download; app AI driver di ricavi importante.
- Concentrazione: 20 “grandi” sviluppatori incassano 50% degli utili.

1.4 Aspetti Legali, Economici e Garanzie

Copyright e Licenze

- Il software è **opera dell'ingegno** protetta da copyright (non brevettabile in Europa).
- Copia abusiva = reato penale in Italia (L. 248/2000: 6 mesi – 3 anni carcere).
- **Licenze:** Proprietaria / Public Domain / Open Source (GPL, MIT, Apache, BSD...).
- **Dibattito 2025:** di chi è il codice generato da LLM (Copilot, Claude)? I modelli “open-weight” (es. Llama) sono davvero open source?

SIAE e Decompilazione. SIAE: registro pubblico per software originali/creativi; deposito su supporto non riscrivibile. **Decompilazione:** generalmente vietata dalle licenze proprietarie; eccezioni limitate per interoperabilità (direttiva UE 2009/24).

Garanzie: La Regola Pratica

Il software commerciale viene venduto “**as is**” (**così com'è**), **senza garanzie**. Il contratto (EULA) esclude garanzie di commerciabilità, idoneità, assenza virus, accuratezza. Tutto il rischio è sull'utente. **Termini tecnici:**

- **Verifica:** aderenza alla specifica (“abbiamo costruito il prodotto giusto?”).
- **Validazione:** accettazione dal cliente (“abbiamo costruito il prodotto che serve?”).
- **Certificazione:** aderenza a specifiche di legge/standard.

1.5 Rischi e Qualità del Software

Rischi del Software:

1. Rischi di sviluppo

2. Difetti nel software operativo
3. Rischi di esercizio
4. **Allucinazioni durante generazione AI** (nuovo, 2025+)

Casi Studio Recenti

- **CrowdStrike, 19 luglio 2024:** aggiornamento difettoso Falcon Sensor → 8,5 milioni di sistemi Windows bloccati, ≥ 10 miliardi \$ danni globali. Settori: aerei, banche, ospedali, emergenze, governi.
- **NATS UK, 15 settembre 2026:** difetto nel National Airspace System → 3.000 voli bloccati, 9 aeroporti, 150.000 passeggeri. Terzo guasto rilevante in 3 anni (non attacco informatico).

Qualità e Difettosità (Difetti/KLoC):

Contesto	Difetti/KLoC
Peggiori sistemi militari	55
Migliori sistemi militari	5
Sviluppo Agile (XP)	1,4
Apache Web Server (open source)	0,5
NASA Space Shuttle	0,1

Codice Generato da AI: Allarme Sicurezza (Dati 2025)

- **Veracode 2025** (100+ LLM testati): **45%** del codice generato contiene vulnerabilità.
- **CodeRabbit (dic. 2025):** codice AI ha probabilità $1,9\text{--}2,7\times$ maggiore di introdurre password deboli, XSS, deserializzazione insicura; **$2,74\times$** più vulnerabilità del codice umano.
- **Apiiro (giu. 2025):** >10.000 nuove segnalazioni sicurezza/mese in codice AI-generato, $10\times$ vs dic. 2024.

1.6 Evoluzione dell'AI e Visione della Disciplina

Timeline Gartner:

- 2022: Foundation Models
- 2023: Generative AI
- 2024: AI Engineering
- 2025: Agentic AI
- 2026: Orchestratori

Avvertimento Gartner

Rischio di “agent-washing”; si prevede che **>40% dei progetti agentici verrà abbandonato entro il 2027** per mancanza di valore dimostrato. Il 2026 sarà anche anno di disillusione e consolidamento.

Studio METR 2025 (RCT: 16 sviluppatori esperti, 246 task reali):

- Con strumenti AI (Cursor + Claude): sviluppatori **19% più lenti**, non più veloci.
- Previsione pre-task: +24% velocità; percezione post-task: +20% velocità. *Percezione ≠ realtà misurata.*
- Calo maggiore su codice grande e ben noto – proprio dove l’AI dovrebbe aiutare di più.

Impatto su Stack Overflow. Dal lancio di ChatGPT (nov. 2022), il volume mensile di domande su Stack Overflow è **crollato**: caso di scuola su come i LLM abbiano cambiato il modo di cercare aiuto nello sviluppo.

Visione dell’Ingegneria del Software: Principi Guida

- **Software = infrastruttura cruciale** per ogni attività umana/tecnologica; complessità inimmaginabile.
- **Innovazione continua**: adattarsi e imparare costantemente → competenza chiave.
- **Agile & DevOps**: processi collaborativi decisivi per dominare la complessità.
- **Responsabilità etica**: il software non è neutrale; considerare impatto sociale.
- **Ingegneria AI-augmented**: agenti conversazionali affiancano lo sviluppatore in scrittura, test, revisione. **Cambia il modo di lavorare, non l’esigenza di verifica, responsabilità e comprensione profonda.**

Aspetti Sociali:

- Sviluppo = lavoro di collaborazione in team interdisciplinari; comunicazione, empatia, prospettive diverse sono cruciali.
- Software costruito *per le persone*: UX, feedback continuo, diversità/inclusione → soluzioni più innovative e inclusive.
- Il software **non è solo codice**: è una costruzione sociale che influenza e viene influenzata da chi lo sviluppa e lo usa.

1.7 Risorse Utili

- stackoverflow.com (volume domande crollato post-ChatGPT)
- www.freelancer.com

- best-practice-software-engineering.blogspot.com
- Gruppi LinkedIn per software developers
- catless.ncl.ac.uk/Risks/ (database rischi software)
- informationisbeautiful.net/visualizations/million-lines-of-code/

2 Essence

2.1 Origini della Disciplina e Iniziativa SEMAT

Rispetto all'informatica pura, che vanta basi teoriche e matematiche solide e consolidate fin dai lavori di Turing e dalla teoria degli algoritmi, l'**ingegneria del software è una disciplina giovanissima**. Nata in modo strutturato solo tra la fine degli anni '60 e gli anni '70 (prima ci si limitava a scrivere codice senza porsi problemi metodologici), l'ingegneria del software ha sempre sofferto della mancanza di presupposti teorici comuni e condivisi.

Questa assenza di fondamenta unificate ha provocato una proliferazione incontrollata di approcci: statistiche industriali contano **oltre un migliaio di metodologie e ricette differenti**. Spesso si tratta di variazioni minime dello stesso concetto sviluppate da singole persone o aziende, che utilizzano vocaboli diversi per descrivere le medesime pratiche o vocaboli identici per descrivere procedure sottilmente disallineate.

I Quattro Sintomi Critici della Frammentazione

- **Babele terminologica e attriti nel turnover:** quando persone con formazioni diverse entrano nello stesso team, o quando un membro se ne va a metà progetto e viene sostituito, si sprecano intere settimane solo per far capire al nuovo arrivato a che punto si trovi il lavoro e di cosa si stia parlando.
- **Cultura dei documenti vs cultura dei risultati:** per difendersi dalla complessità e dai costi crescenti, le organizzazioni (specie le grandi realtà come Google o Microsoft rispetto alle piccole software house da 6–7 persone) hanno sviluppato una pesante burocrazia documentale. Tuttavia, documentare meticolosamente ogni passo è statisticamente l'attività meno gradita agli sviluppatori, che preferiscono scrivere codice; il risultato è una montagna di documentazione “write-only” che finisce presto dimenticata e cestinata.
- **Pratiche non testate sul campo:** la maggior parte di queste ricette metodologiche non è mai stata sottoposta a reale validazione empirica. La loro efficacia resta discutibile e produce esiti disomogenei: anche un professionista esperto in una specifica pratica rischia di fallire quando si sposta in un gruppo che lavora in modo simile ma non identico.
- **Il dilemma dell'insegnamento:** per i docenti è complicato insegnare la materia senza dover “battezzare” arbitrariamente un metodo specifico (es. Scrum, XP) a scapito di altri. Essence risolve questo nodo fornendo un quadro di riferimento neutro a monte.

L'Iniziativa SEMAT e la Genesi di Essence. Per superare lo scontro ideologico tra metodologie contrapposte e dotare la disciplina di solide fondamenta scientifiche, nel 2009 prende vita l'iniziativa internazionale **SEMAT** (*Software Engineering Method and Theory*).

Tappe Fondamentali di SEMAT

- **2009 – Fondazione SEMAT:** promossa da tre figure di spicco dell'informatica accademica e industriale: **Ivar Jacobson** (co-ideatore di RUP e pioniere dei casi d'uso), **Bertrand Meyer** (creatore di Eiffel e del paradigma Design by Contract) e **Richard Soley** (storico presidente di OMG). L'obiettivo primario è definire una teoria rigorosa per l'ingegneria del software.
- **2012 – Prima Specifica del Kernel:** pubblicazione della prima specifica ufficiale del Kernel di Essence, elaborata attraverso una collaborazione internazionale e validata su casi d'uso concreti.
- **2014 – Adozione come Standard OMG:** Essence diventa formalmente uno standard dell'**Object Management Group** (lo stesso consorzio di standardizzazione internazionale di UML e BPMN), sancendone il riconoscimento universale.

2.2 Cos'è Essence e i 7 Alpha del Kernel

Lungi dall'essere un puro formalismo teorico o un'ennesima sovrastruttura burocratica, Essence è prima di tutto uno **strumento orientato alla conversazione**, elemento che costituisce il vero filo conduttore dell'intero corso. Serve a consentire a persone con background e ruoli diversi di comunicare efficacemente sullo stato di un progetto software senza fraintendimenti.

Metamodello vs Metodo. Un equivoco comune consiste nel catalogare Essence come un'ulteriore metodologia concorrente (come Scrum o XP). In realtà Essence si colloca a un meta-livello:

I Tre Pilastri Concettuali

- **Non è un metodo:** non impone un processo prescrittivo rigido, né una sequenza temporale di attività da eseguire alla cieca.
- **È un metamodello:** descrive la struttura invariante e il vocabolario fondazionale comune a qualsiasi metodo di sviluppo software.
- **È universale e agnostico:** è applicabile trasversalmente a paradigmi differenti, integrandosi con Scrum, Kanban, RUP, SAFe o processi interni personalizzati.

Principio Chiave: Diagnosi vs Cura

Il Kernel di Essence svolge una funzione prettamente **diagnostica, non prescrittiva**:

- **Fornisce la diagnosi:** rileva con precisione lo stato di avanzamento, le incongruenze e i blocchi del progetto (opera come un termometro oggettivo).
- **Non prescrive la cura:** il Kernel non dice al team *come* risolvere il problema emerso o quali attività specifiche implementare.

- **Responsabilità del Team:** decidere quale azione correttiva intraprendere per superare un ostacolo e transitare allo stato successivo rimane una precisa responsabilità del team, che sceglierà le pratiche operative opportune.

Essence si compone fondamentalmente di due elementi:

1. Un **Kernel comune**: un insieme essenziale e minimale di concetti universali sempre presenti in ogni sforzo ingegneristico.
2. Un **Linguaggio universale**: un formalismo grafico e testuale per formalizzare, comparare, comporre e adattare pratiche ingegneristiche modulari.

I 7 Alpha e il Ruolo degli Esiti. Il Kernel individua 7 elementi cardine, denominati **Alpha** (*essential things to work with*), che rappresentano gli aspetti vitali da far progredire in qualsiasi iniziativa software.

Cambio di Paradigma

La novità sostanziale risiede nella misurazione del progresso: non si valuta più la conformità tramite la mera consegna di documenti formali, bensì il raggiungimento di **esiti misurabili e oggettivi** (*outcomes*). Ogni Alpha progredisce attraverso una sequenza definita di **stati di salute**, ciascuno con precisi criteri di soddisfazione verificabili.

Nel panorama generale di un progetto possono esistere molti più elementi da monitorare, e singole pratiche specialistiche possono introdurre o specializzare ulteriori Alpha (o sotto-Alpha). Tuttavia, i **7 Alpha del Kernel** rappresentano il minimo comune denominatore invariante: sono quegli aspetti basilari che risultano presenti *sempre e comunque* in qualunque progetto software.

Le Tre Aree di Interesse (Areas of Concern). Per offrire una visione d'insieme chiara e immediata, i 7 Alpha sono raggruppati in tre macro-aree ortogonali, codificate per colore:

Area di Interesse	Alpha Inclusi	Finalità e Focus
Customer (<i>Il Cliente</i>)	Opportunità Stakeholder	Rappresenta il valore per utenti e committenti. Assicura che l'iniziativa risponda a reali bisogni di mercato o di business e generi un impatto effettivo.
Solution (<i>La Soluzione</i>)	Requisiti Sistema Software	Riguarda la progettazione e la realizzazione tecnica. Garantisce che la specifica sia coerente e che il software evolva sotto controllo architetturale.
Endeavor (<i>L'Iniziativa</i>)	Lavoro Team Modo di Lavorare	Governa le persone, l'organizzazione e le attività. Assicura che le modalità di collaborazione e i flussi operativi siano sostenibili ed efficaci.

Descrizione dei 7 Alpha:

1. **Opportunità (*Opportunity*):** il valore di business e le circostanze di mercato o strategiche che giustificano l'investimento e lo sviluppo del prodotto.
2. **Stakeholder:** le persone, i gruppi o le organizzazioni interessati all'esito del sistema, i cui requisiti e feedback determinano il successo del progetto.
3. **Requisiti (*Requirements*):** cosa il sistema software deve fare e quali vincoli/qualità deve rispettare.
4. **Sistema Software (*Software System*):** il manufatto tangibile in costruzione, comprensivo di codice, test, configurazioni e documentazione operativa.
5. **Lavoro (*Work*):** la suddivisione delle attività, i task e l'impegno operativo necessario per raggiungere gli obiettivi prefissati.
6. **Team:** il gruppo di persone che collabora attivamente alla realizzazione delle attività di progetto.
7. **Modo di Lavorare (*Way of Working*):** l'insieme delle pratiche, convenzioni, accordi interni e strumenti adottati dal team per cooperare.

2.3 Strumenti Pratici, Metodologie e Formazione

Una peculiarità di Essence è la traduzione del Kernel in un formato fisico: **mazzi di carte da gioco**. Questo consente sia una consultazione rapida da parte dei singoli professionisti, sia la facilitazione di dinamiche partecipative strutturate all'interno del team.



Figura 1: I 7 Alpha del Kernel di Essence e la suddivisione nelle tre Aree di Interesse.

Tecniche e Serious Games

- **Progress Poker:** ciascun membro valuta indipendentemente lo stato corrente di un Alpha giocando la carta corrispondente; le discrepanze emerse stimolano il confronto per giungere a una stima condivisa e oggettiva.
- **Milestone Mapping:** il team stabilisce le milestone progettuali definendo con precisione quali stati di quali Alpha devono essere soddisfatti per il traguardo, ottenendo una roadmap priva di ambiguità.
- **Checkpoint:** sessioni periodiche di audit per esaminare lo stato di salute generale del sistema attraverso le checklist degli stati degli Alpha.

Integrazione con le Metodologie Esistenti. Essence non intende sostituirsi ai framework agili o prescrittivi esistenti (Scrum, Kanban, SAFe), ma si propone come loro integratore sistemico:

Valore Aggiunto di Essence sulle Metodologie

- **Integrazione, non sostituzione:** formalizza con un vocabolario neutro ciò che una metodologia prescrive e ne evidenzia i confini (es. Scrum struttura il *Lavoro* e il *Team*, ma non fornisce pratiche per l'architettura del *Sistema Software*).
- **Identificazione delle lacune:** consente di diagnosticare tempestivamente quali Alpha sono trascurati all'interno dell'organizzazione.
- **Pratiche componibili come building blocks:** permette di trattare le pratiche inge-

gnéristiche (es. User Stories, Standup, TDD, Code Review) come componenti modulari combinabili a piacere per definire un processo su misura (*tailored*).

Essence nella Formazione Universitaria. Per uno studente di ingegneria del software, studiare Essence fornisce vantaggi strutturali rispetto all'apprendimento mnemonico di un singolo processo:

1. **Pensiero critico sui processi:** capacità di analizzare qualunque metodologia chiedendosi cosa copra, cosa ometta e se sia adatta allo specifico contesto operativo.
2. **Risoluzione strutturata dei problemi:** adozione di una griglia diagnostica formale per localizzare dove e perché un progetto manifesti criticità (es. mancato allineamento degli *Stakeholder* vs inefficienze nel *Modo di Lavorare*).
3. **Adattabilità al contesto industriale reale:** preparazione a operare in contesti aziendali eterogenei e a partecipare attivamente all'evoluzione dei processi di sviluppo.

2.4 Metodi, Pratiche e Ciclo di Vita degli Alpha

L'esperienza industriale dimostra come l'adozione acritica di metodologie formalizzate spesso non conduca ai benefici sperati. I fattori determinanti di tale inefficacia includono:

- **Elefantiasi prescrittiva:** i metodi tradizionali tendono ad accumulare nel tempo una sovrabbondanza di ruoli specialistici, artefatti documentali e cerimonie burocratiche che appesantiscono il flusso di lavoro.
- **Adozione nominale (Cargo Cult):** le organizzazioni adottano sovente la terminologia esteriore di una metodologia (es. chiamare le riunioni *Standup* o i requisiti *User Story*), senza modificare la sostanza delle pratiche operative sottostanti.
- **Discrepanze terminologiche:** metodologie differenti impiegano vocaboli differenti per descrivere le medesime problematiche essenziali, ostacolando il confronto oggettivo e il riuso della conoscenza.
- **Focus sugli artefatti vs focus sugli esiti:** i report di avanzamento convenzionali misurano la quantità di documenti prodotti anziché i risultati concreti (*outcomes*) e il reale stato di salute del progetto.

La Risposta di Essence

Essence affronta queste criticità introducendo una netta separazione delle preoccupazioni:

- **Linguaggio comune minimale:** stabilisce un terreno condiviso (*common ground*) per tutta l'ingegneria del software.
- **Separazione tra universale e contingente:** isola i concetti invarianti di qualsiasi iniziativa rispetto alle guide operative specifiche delle singole pratiche.
- **Progresso visibile e basato su evidenze:** sostituisce la conformità documentale con stati oggettivi e checkpoint verificabili.

- **Composizione ed evoluzione autonoma:** consente ai singoli team di assemblare e far evolvere il proprio modo di lavorare su misura.

Tassonomia: Processo, Metodo e Pratica. Per comprendere l'architettura di Essence è indispensabile distinguere nettamente tre concetti spesso confusi in letteratura:

Concetto	Definizione e Ruolo Operativo
Processo (<i>Process</i>)	Descrive un flusso ordinato di lavoro nel tempo , specificando sequenze, transizioni e dipendenze temporali tra attività.
Metodo (<i>Method</i>)	Fornisce a un team una guida complessiva e strutturata per condurre un'iniziativa (<i>endeavour</i>) dal principio alla conclusione.
Pratica (<i>Practice</i>)	Approccia e risolve un aspetto specifico del lavoro (es. stima a punti, retrospettive, TDD, code review); è modulare e componibile con altre pratiche.

I Metodi come Composizione di Pratiche

In Essence, **un metodo non è un monolite, ma una composizione modulare di pratiche.**

Questa visione poggia su tre presupposti cardine:

1. L'ingegneria del software possiede un **terreno comune invariante** (*Kernel*).
2. I metodi sono assemblati selezionando e coordinando singole pratiche specialistiche.
3. I team devono concentrarsi sull'**essenziale** e sulla raccolta di **evidenze oggettive**.

Metafora dell'Albero: Il Kernel rappresenta l'albero (la struttura portante comune e stabile), mentre le pratiche costituiscono le decorazioni e le luci che ogni organizzazione sceglie di appendere per personalizzare il proprio modo di lavorare.

Anatomia di un Alpha: Cicli di Vita, Stati e Checkpoint. Un Alpha non è un documento, un deliverable o un modello. È una **preoccupazione essenziale** (*essential concern*) che possiede:

- Un **ciclo di vita esplicito** articolato attraverso una sequenza ordinata di **stati di salute**.
- Una natura agnostica rispetto alla documentazione: **più prodotti di lavoro** (*Work Products*) **differenti** possono concorrere a fornire l'evidenza necessaria per uno stesso Alpha.
- Universalità: mantiene il suo significato invariato indipendentemente dal metodo adottato (sia esso Waterfall, Scrum o XP).

Ciascun passo nel ciclo di vita di un Alpha è definito da due costrutti precisi:

- **Stato** (*State*): condizione significativa e verificabile lungo il ciclo di vita dell'Alpha.

- **Checkpoint:** elenco rigoroso delle evidenze tangibili richieste per certificare il raggiungimento dello stato corrispondente.
- **Proprietà cumulativa:** uno stato successivo incorpora intrinsecamente il soddisfacimento di tutti gli stati precedenti (*a later state includes the intent of earlier states*).

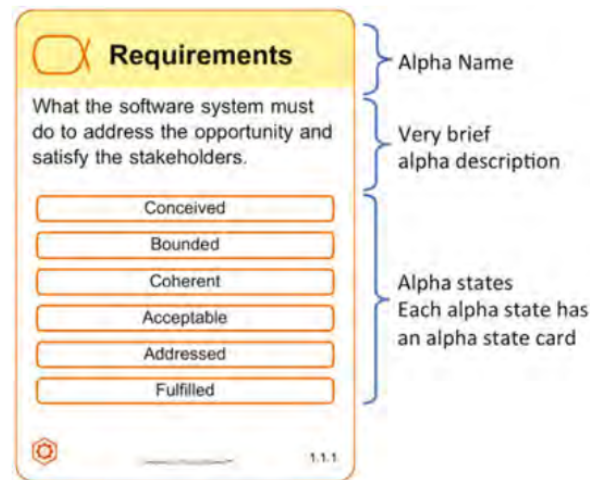


Figura 2: Anatomia di una carta Alpha di Essence e gerarchia degli stati con checklist.

I 7 Alpha del Kernel e i Relativi Stati Ordinati

1. Customer:

- **Opportunity:** *Identified → Solution Needed → Value Established → Viable → Addressed → Benefit Accrued.*
- **Stakeholders:** *Recognized → Represented → Involved → In Agreement → Satisfied for Deployment → Satisfied in Use.*

2. Solution:

- **Requirements:** *Conceived → Bounded → Coherent → Acceptable → Addressed → Fulfilled.*
- **Software System:** *Architecture Selected → Demonstrable → Usable → Ready → Operational → Retired.*

3. Endeavour:

- **Work:** *Initiated → Prepared → Started → Under Control → Concluded → Closed.*
- **Team:** *Seeded → Formed → Collaborating → Performing → Adjourned.*
- **Way of Working:** *Principles Established → Foundation Established → In Use → In Place → Working Well → Retired.*

Relazioni intercorrenti tra gli Alpha. Gli Alpha non procedono in isolamento, ma interagiscono dinamicamente:

- L'**Opportunity** guida ed orienta la specifica dei **Requirements**.
- Gli **Stakeholders** utilizzano, valutano e supportano il **Software System**.
- Il **Team** pianifica, coordina ed esegue il **Work**.
- Il **Way of Working** disciplina e sostiene contestualmente sia il **Team** sia il **Work**.

Progresso vs Salute del Progetto. Nell'ingegneria del software, Valore, Soluzione e Iniziativa evolvono congiuntamente ma a velocità spesso disomogenee. Le metriche di avanzamento tradizionali (es. *burndown chart*, ore lavorate, task completati) tendono a mascherare anomalie critiche: un progetto può apparire perfettamente in orario mentre accumula un debito di rischio insostenibile su un Alpha trascurato.

Progresso vs Salute (Health)

- **Progresso (*Progress*)**: misura quantitativa di quanto un singolo Alpha sia avanzato lungo la sua sequenza di stati di salute.
- **Salute (*Health*)**: valutazione qualitativa sull'idoneità dello stato attuale dell'Alpha a sostenere l'iniziativa senza compromettere gli altri elementi del Kernel.
- **Ostacoli al prossimo stato (*Next-state obstacles*)**: l'attenzione diagnostica del team non deve limitarsi a certificare lo stato raggiunto, ma deve identificare tempestivamente le barriere che impediscono il soddisfacimento dei checkpoint dello stato immediatamente successivo.

2.5 Elementi Complementari e Studio di Caso (UniMagic)

Work Products vs Alpha. Un **Work Product** è un artefatto tangibile e contingente (es. il Product Backlog di Jira, un diagramma delle classi, una build eseguibile, una specifica di collaudo).

- La produzione di un Work Product **non implica automaticamente l'avanzamento di stato dell'Alpha**. Ad esempio, scrivere un documento di 200 pagine non fa transitare i *Requirements* allo stato *Coherent* se persistono contraddizioni o se gli stakeholder non hanno validato i casi d'uso critici.
- L'artefatto assume valore ingegneristico esclusivamente quando fornisce l'**evidenza** idonea a soddisfare i checkpoint dell'Alpha corrispondente.

Attività e Spazi di Attività (Activity Spaces):

- **Spazio di Attività (*Activity Space*)**: definisce un'area di lavoro universale e invariante prevista dal Kernel (il "*cosa fare*" ad alto livello, es. *Explore Possibilities*, *Coordinate Activity*).

- **Attività (*Activity*)**: rappresenta l'unità esecutiva di lavoro definita da una specifica pratica adottata (il “*come farlo*”, es. *Sprint Planning*, *Code Review*).
- Il mapping tra le attività concrete e gli Activity Spaces del Kernel consente di identificare immediatamente duplicazioni, ridondanze o lacune metodologiche nell'organizzazione.

Competenze e Pattern:

- **Competenze (*Competencies*)**: abilità, conoscenze ed esperienze formali richieste alle figure umane per condurre specifici compiti ingegneristici.
- **Pattern**: soluzioni standardizzate e riutilizzabili a problemi ricorrenti. In questo metamodello, i **Ruoli** (es. Scrum Master, Product Owner, Tech Lead) sono formalizzati come pattern composti che aggregano insieme coerenti di responsabilità e competenze.

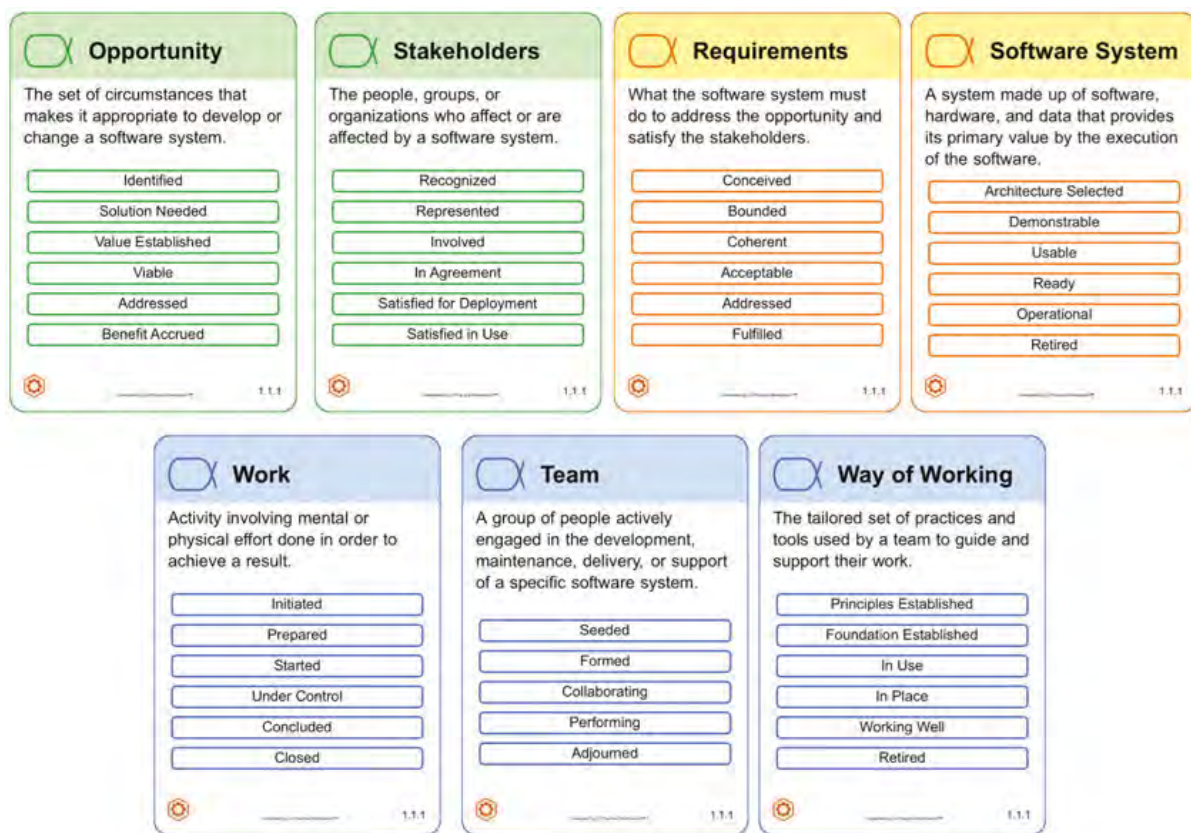


Figura 3: Anatomia di una carta Alpha di Essence e gerarchia degli stati con checklist.

Studio di Caso Applicativo: Diagnosi del Progetto UniMagic. Una software house fondata di recente da circa 12 persone (prevalentemente sviluppatori web) riceve il suo primo incarico commerciale significativo dall'*Università di Warthog*. Il progetto consiste nello sviluppo di **Uni-Magic**, una piattaforma digitale per unificare la gestione del supporto ai corsi, la prenotazione degli appuntamenti e le istanze amministrative degli studenti.

Vincoli e Condizioni di Partenza:

- Rilascio della prima versione operativa a **2.000 studenti** entro un semestre accademico.

- Divergenze e disaccordo tra gli stakeholder in merito alle priorità funzionali e ai vincoli di privacy.
- Team di sviluppo composto da 6 persone, con esperienza in ambienti web ma completamente prive di competenze su Essence.

Diagnosi degli Alpha dello Scenario (Senza Prescrizione di Cura)

- **Requirements** → **Stato massimo:** *Conceived*: L'ambito generale del servizio è stato delineato, ma la persistenza di conflitti non sanati sulle priorità e sulle policy di riservatezza impedisce di considerare i requisiti circoscritti e condivisi. Non è soddisfatto il checkpoint di confine per lo stato *Bounded*.
- **Team** → **Stato effettivo:** *Formed*: I 6 membri sono stati identificati e assegnati al progetto, ma operano come somma di individualità. Mancano le evidenze oggettive di cooperazione coesa, fiducia reciproca e allineamento sulla missione richieste per lo stato *Collaborating*.
- **Way of Working** → **Stato effettivo:** *Principles Established*: Il gruppo ha espresso l'intenzione di adottare uno sviluppo iterativo, ma non ha ancora istanziato né validato sul campo strumenti e pratiche operative concordate. L'approccio non è ancora operativamente *In Use*.

Analisi delle criticità e delle asserzioni non supportate:

- **Asserzione a rischio:** l'assunzione che il primo rilascio sia pienamente fattibile per 2.000 utenti entro un solo semestre.
- **Evidenza mancante:** non sussistono evidenze relative all'architettura tecnica del *Software System*, alla capacità produttiva misurata del *Team* e alla convergenza contrattuale degli *Stakeholders*. Il progetto presenta un elevato rischio di fallimento latente non rilevabile dai soli diagrammi di pianificazione.
- **Ruolo del Team nella Soluzione:** Essence ha diagnosticato il disallineamento; spetta ora al team definire la strategia risolutiva (es. rinegoziare lo scope con il cliente, introdurre pratiche di prototipazione rapida o adottare spike architetturali).